

WEEK 01

<https://drive.google.com/file/d/1CnHq7YC8Y5--NP-N4zXPTLaiqD4gs7q4/view>

<https://drive.google.com/file/d/1QIsfXOgtXAs5KabYgEoccyIuf7L8u02j/view>

<https://drive.google.com/file/d/18JcthGPvpSwkEpFPXwflyf-t7V8-mz-r/view>

GPT LINK: <https://chatgpt.com/share/697f41f4-1284-800c-96ea-967a6c33f0a0>

★ information retrieval:

- Process of retrieving relevant documents in correspondence to a query
- The activity of obtaining **information resources (docs)** relevant to an **info need(query)** from a collection of information resources (docs) stored on web/computers
- Finding material (docs) of an unstructured nature (text) that satisfies an info need (query) from within large digital collections (stored on computer/web)

★ data / information / knowledge / wisdom



DIKW Pyramid

⇒ **DATA:** raw representation, symbols/signs

⇒ **INFO:** processed data, useful to answer questions, data in context, linked data

⇒ **KNOWLEDGE:** organized info that can be acted upon

⇒ **WISDOM:** applied knowledge

★ database vs IR

DATABASE	INFORMATION RETRIEVAL
1) Retrieves Structured data 2) Query: SQL → unambiguous 3) Result: exact n correct 4) Single (periodic) query / Analytical (case specific) query	1) Retrieves mostly unstructured data 2) NLP → imprecise 3) Result is relevant/ related 4) Interactive interaction 5) Data is READ-ONLY, no update

★ NLP – Natural language processing

- NLP : teaching computers to understand,interpret and generate human language
- A word and **what meaning** can be extracted from it matters !!!
- IR uses NLP to get better results [Retrieves semantically relevant documents]

👉 Early IR systems were keyword-based:

- Exact word match
- No understanding of meaning

👉 Modern IR systems are NLP-powered:

- Understand synonyms
- Understand intent
- Understand context

★ Text Mining vs. Data Mining

Text Mining	Data Mining
-------------	-------------

1) Extracting hidden/meaningful information from unstructured/semi-structured text data 2) Input: unstructured/semi-structured 3) NLP, IR and linguistic analysis used 4) Challenges : handling ambiguity, context understanding and natural language variations 5) Involves tokenization, stop word removal, stemming, lemmatization!	1) Discovering patterns,trends and knowledge from structured data stored in db 2) Input: structured numerical or categorical data 3) Statistical method/ML/db queries used 4) Challenges: managing large db, ensuring data quality, computational scalability 5) Involves data cleaning, normalization and transformation!
---	---

★ stemming vs. lemmatization

- **text normalization** techniques in NLP.
- Reduce words to a **base form** so different variations are treated as the same word.

 Stemming	Lemmatization
- Stemming cuts off word endings using simple rules. • No understanding of meaning • Just chops suffixes/prefixes - ❌ Not linguistically accurate - ✅ Very fast - ✅ Good for large-scale search systems	- Lemmatization converts words to their dictionary form (lemma) using: • Vocabulary • Grammar (part of speech) • Context - 📌 Lemmatization always gives real words - ❌ Slower than stemming - ❌ Needs POS tagging [Labeling each word in a sentence with its grammatical role]

Feature	Stemming	Lemmatization
Uses rules	✓	✗
Uses dictionary	✗	✓
Output real words	✗	✓
Speed	🚀 Fast	🐢 Slower
Accuracy	Low	High
Context aware	✗	✓

🚀 What should *you* use?

Use Stemming when:

- Building a **search engine**
- Speed matters more than accuracy
- Large-scale text processing

Use Lemmatization when:

- Doing **sentiment analysis**
- NLP research
- Any task where meaning matters

★ Text Mining vs. Information Retrieval

Aspect	Text Mining	Information Retrieval
Definition	The process of extracting meaningful insights and patterns from unstructured or semi-structured text data.	The process of retrieving relevant documents or information from a large repository in response to a query .
Input Data	Unstructured or semi-structured text data (e.g., documents, emails, articles).	Structured or unstructured data repositories, such as databases, web pages, or document collections.
Objective	Derive hidden patterns, insights, or relationships from text.	Find and rank the most relevant documents for a given query.
Techniques Used	Natural Language Processing (NLP), machine learning, and statistical analysis.	Search algorithms, indexing, ranking, and relevance feedback.
Data Preprocessing	Tokenization, stemming, lemmatization, and stop-word removal.	Index creation, keyword extraction, and document ranking.
Challenges	Handling language ambiguity, context, and polysemy.	Balancing precision and recall, handling diverse queries, and scalability.

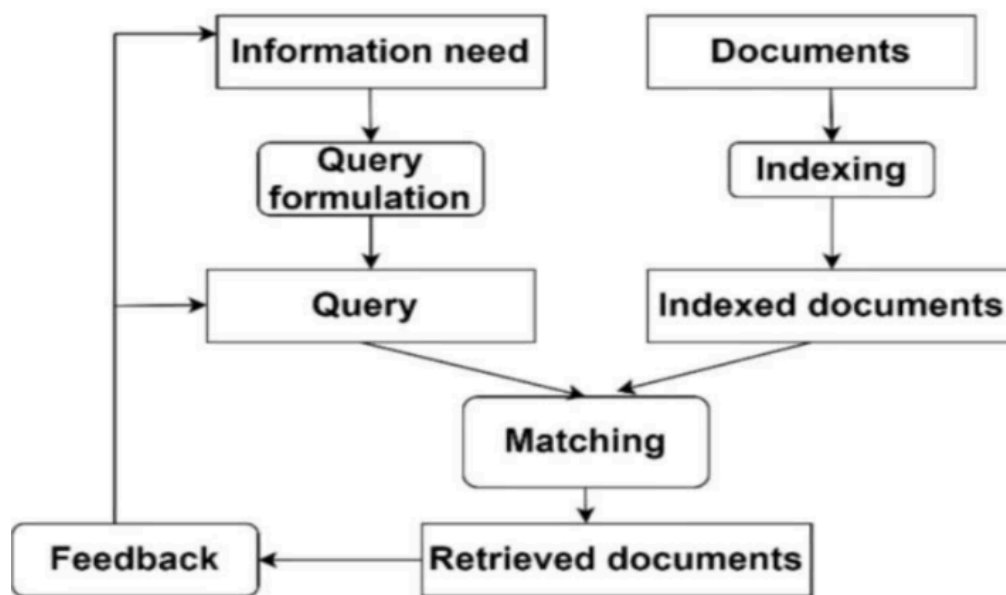
★ Adhoc Retrieval Systems

An **Ad-hoc Retrieval System** is an **Information Retrieval (IR)** system that:

- Retrieves documents from a **fixed collection**

- Based on an **arbitrary (unknown in advance) user information need (query)**
 - Expressed through a **one-time (one-off) query**
 - The system has **no prior knowledge** of the user's intent
-
- Google , bing (later became a decision engine)
 - repository/resource available that searches for our query for relevant results
 - Ranks the results
 - Info repo needed, shd know which repo satisfies our request!

★ Basic Information Retrieval Process



★ Content-wise IR systems

⇒ Content-wise classification groups IR systems by **what they search, how big the collection is, how often it changes, what type of content it contains, and how strict the matching is.**

1) Contents by SIZE (collection scale)

a) Personal / Desktop Search

- Searching ur own files : docs,pdfs,emails,notes,app content
- **Collection size:** small to medium (thousands to millions of files max, but locally)
- **Goal:** extremely fast retrieval, often “as you type”
- **Features:** indexing happens on your device; supports file metadata (date, type, author), content search inside PDFs/docs
- **Example:** Windows Search

b) Web-Scale search

- Searching the public web (billions of pages)
- **Collection size:** massive (web-scale)
- **Main challenge:** ranking quality (showing the *best* results first) + spam handling + freshness
- **Features:** crawling, indexing, link analysis, semantic understanding, personalization (sometimes), location/language awareness
- **Example:** google, DuckDuckGo

c) Enterprise Search

- Searching a **restricted or specialized corpus** (company docs, legal databases, medical repositories, insurance/policy portals, etc.)
- **Collection size:** medium to large, but within an organization/domain
- **Main challenge:** correctness + permissions + specialized vocabulary
- **Features:** access control (who can see what), structured data integration, compliance/audit trails, domain ranking rules
- **Examples:** Westlaw , Policybazaar

⇒ Note: Enterprise search isn't always "bigger than web search" — it's "bigger than personal" but **more controlled and more specialized** than web.

2) Contents by change over time

a) Static (Offline) collections

- document collection is **mostly fixed** (or updated rarely).
- You can index it once and search repeatedly.
- **Examples:** archived research papers, a fixed dataset, a downloaded encyclopedia
- **Benefits:** faster, stable ranking, easier evaluation
- **Downside:** doesn't reflect new information quickly

b) Dynamic (Online) collections

- collection is **constantly changing**.
- **Examples:** news pages, social media, product catalogs, web pages that update, company docs that change daily
- **Challenges:**
 - keeping the **index fresh** (new docs appear, old docs disappear)
 - "freshness" as a ranking signal (today's news vs 2019 article)
 - duplicates and near-duplicates

3) Contents by type of data:

a) Text IR

- queries and documents are text.
- **Query:** keywords or natural language
- **Documents:** web pages, PDFs, emails, articles
- **Tech:** TF-IDF/BM25, embeddings, transformers, query expansion, spelling correction

b) Multimedia IR

- Searching **non-text** content: images, audio, video.
- **Tech:** feature vectors/embeddings from vision/audio models; metadata + captions often help

c) Mixed (multi-modal) IR

- Combination of text + multimedia
- Mixed IR often uses:
 - **text signals** (title, description)
 - **visual signals** (image similarity)
 - **behavioral signals** (clicks, purchases)
 - **structured fields** (brand, price)

4) Exact Match vs Best Match (matching strategy)

a) exact Match

- Results only appear when there is a **strict match** (or near strict) between query and document terms/fields.
- **Where it's used:**
 - **database lookup** (IDs, roll numbers)
 - **filters** ("category = shoes AND size = 42")
 - legal/medical cases where exact wording matters sometimes
- **Pros:** precise, predictable
- **Cons:** misses relevant results if wording differs (synonyms, typos, paraphrases)

b) Best match

- Ranked retrieval
- System returns results that are **most relevant**, even if they don't contain the exact same words.
- **How it works:** documents get a relevance score, then sorted (ranked)
- **Signals used:**
 - **keyword similarity** (TF-IDF/BM25)
 - **semantic similarity** (embeddings)

- freshness, popularity, links, authority
 - personalization/context (sometimes)
- **Pros:** better user experience, handles synonyms/paraphrases
 - **Cons:** can return “kinda related” results; ranking can be harder to explain

Content	Applications	Tasks
Text	Web search	Ad hoc search
Images	Vertical search	Filtering
Video	Enterprise search	Classification
Scanned docs	Desktop search	Question answering
Audio	Forum search	Clustering
Music	P2P search	
	Literature search	

✗ This table does NOT mean each row is connected

Column 2: Applications (WHERE is IR used?)

This answers:

“In what kind of system or environment is IR applied?”

Application	Meaning
Web search	Google-like search
Vertical search	Domain-specific (medical, legal, shopping)
Enterprise search	Company/internal documents
Desktop search	Files on your laptop
Forum search	Searching posts/questions
P2P search	Peer-to-peer networks
Literature search	Research papers, journals

Column 3: Tasks (WHAT is the system doing?)

This answers:

"What problem is the IR system solving?"

Task	Meaning
Ad-hoc search	One-time user query
Filtering	Automatically selecting relevant items
Classification	Assigning labels
Question answering	Direct answers
Clustering	Grouping similar items

★ Large Language Model – LLMs

- A **Large Language Model (LLM)** is:
 - An **AI model trained on massive amounts of text**
 - Uses **deep learning**, most commonly **Transformer architecture**
 - Learns **patterns, structure, and relationships in language**
 - Can **understand and generate human-like text**
- **Generative model that can generate a textual response against a prompt**
- An LLM does NOT understand meaning like humans.
- It predicts what text is most likely to come next.

? How does an LLM work internally?

1) How does it start?

- The **prompt** acts as the starting context
- The model converts the prompt into **tokens**
- These **tokens** are fed into the **transformer**

📌 If prompt is empty → special **start token** is used.

2 How does it decide what to write next?

At every step, the model:

- Looks at **all previous tokens**
- Computes probabilities for **every possible next token**
- Chooses one based on probability

👉 This is called **next-token prediction**.

3 "Most probable sequence of text" (important exam phrase)

The model aims to generate: **The most likely sequence of tokens given the prompt**

- $P(w_1, w_2, w_3, \dots, w_n \mid \text{prompt})$
- Each word depends on **all previous words**.

4 How does it stop?

Text generation stops when:

- A **special end-of-sequence token** is produced, OR
- A **maximum length** is reached, OR
- A stop condition (like punctuation or instruction) is met

📌 The model does **not "decide"** to stop intelligently — it follows rules.

📊 Why Transformers?

Transformers allow:

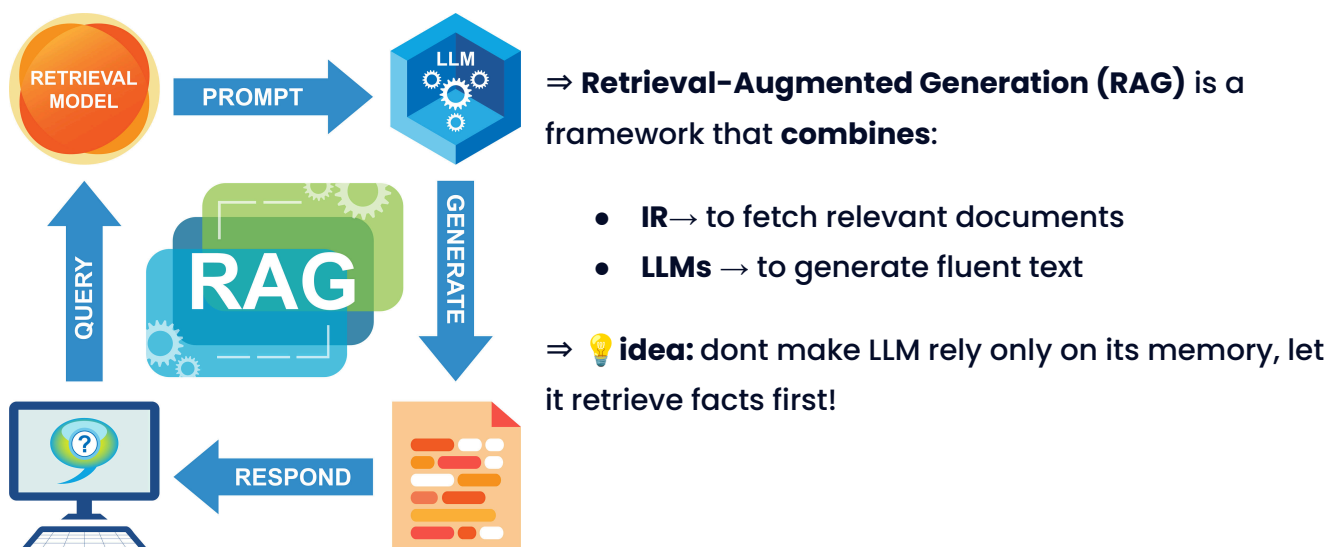
- Attention to **all words at once**
- Understanding long-range dependencies
- Parallel processing (faster training)

Aspect	Large Language Model (LLM)	Information Retrieval (IR)
Functionality	Generates text, interprets queries, and performs reasoning.	Retrieves pre-existing documents or data based on relevance.
Generative Ability	Can create new text (e.g., generate stories or code).	Cannot generate; works only with pre-existing data.
Contextual Understanding	Handles complex linguistic context and reasoning.	Primarily focuses on matching queries with relevant documents.
Applications	Chatbots, text generation, summarization, etc.	Search engines, recommendation systems, etc.
Underlying Models	Based on deep learning (e.g., transformers).	Often uses traditional methods like TF-IDF, BM25, or embeddings.
Dynamic vs Static	Dynamic, as it generates content in real-time.	Static, operating over a fixed dataset.

★ Small Language Models (SLMs)

- A **Small Language Model (SLM)** is:
 - A language model with **fewer parameters**
 - Trained on **smaller or domain-specific datasets**
 - Optimized for **efficiency**, not scale
- Examples: AI chatbot for a specific application, embedded AI

★ Retrieval-Augmented Generation (RAG)



Why do we need RAG?

❌ Problem with pure LLMs • Trained on static data • Can hallucinate (make up facts) • Cannot access new or private knowledge • Answers may sound confident but be incorrect	✅ What RAG fixes • Grounds responses in real documents • Improves factual accuracy • Enables use of private / enterprise data • Keeps the model up-to-date without retraining
--	--

RAG = IR + LLM (bridge explained)

Component	Role
IR system (retriever)	Finds relevant documents
LLM (generator)	Writes a natural-language answer
RAG	Connects both

⚙️ How RAG Works (step by step)

1 Query is input into the Retriever

- User asks a question:

“What are the symptoms of dengue?”

- The query is sent to a **retrieval system**
- The retriever may use:
 - Keyword search (BM25)
 - Vector similarity (embeddings)
 - Hybrid search

✳️ This is **classic IR**.

2 Retriever searches the knowledge base

- Knowledge base can be:
 - PDFs
 - Web pages
 - Research papers
 - Company documents
 - Databases
- Documents are **indexed** beforehand

The retriever finds the **top-k most relevant documents**.

✳️ *Top-k* = best k matches (e.g., k = 3 or 5)

3 Retrieved documents + query → Generator

- The system now creates an **augmented prompt**:

yaml

Question: What are the symptoms of dengue?

Context:

Document 1: Dengue causes high fever, headache...

Document 2: Common symptoms include muscle pain...

- This combined input is sent to the **LLM**

4 Generator produces a grounded response

- The LLM:
 - Reads the retrieved context
 - Uses it as **evidence**
 - Generates a fluent, accurate answer

```
User Query
  ↓
Retriever (IR)
  ↓
Relevant Documents
  ↓
LLM + Context
  ↓
Final Answer
```

🧠 What does “grounded” mean? **Grounded = based on retrieved facts, not guesses**

Feature	RAG	IR
Focus	Generating responses grounded in retrieved knowledge.	Retrieving and ranking relevant documents.
Components	Combines retrieval and generation.	Only retrieval.
Output	Human-like text response.	Pre-existing documents or snippets.
Best Use Case	Complex, conversational, or open-ended queries requiring reasoning.	Straightforward search for existing information.